DEPLOYMENT GUIDE

Deploy Your Logistic Regression Model with GitHub & Streamlit

A step-by-step guide for the customer-purchase model — including how to use your saved encoder files.

model.pkl **city_encoder.pkl** **membership_encoder.pkl**

Raw inputs → Encoders → Model → Prediction

This guide assumes you have already run the training notebook and now have three files: `model.pkl`, `city_encoder.pkl`, and `membership_encoder.pkl`.

About This Model

You trained a **Logistic Regression** model that predicts whether a customer will purchase (`1`) or not (`0`). It uses four features — two numeric and two text:

Feature	Type	Notes
`age`	Numeric	Entered directly as a number.
`income`	Numeric	Entered directly as a number.
`city`	Text	Converted to a number by `city_encoder.pkl` .
`membership`	Text	Converted to a number by `membership_encoder.pkl` .

The key idea of this whole guide: your model only understands numbers. During training, the words in `city` and `membership` were turned into numbers by two encoders, which you saved. In the app you must reload those **exact same** encoders and use them to convert the user's words before predicting. That is what the two `*_encoder.pkl` files are for.

How the Encoder Files Are Used

Before the steps, here is the mental model. Think of each encoder as a small, saved "translation table" that was learned during training:

Encoder file	Remembers this translation
`city_encoder.pkl`	Delhi→0, London→1, Paris→2, Tokyo→3
`membership_encoder.pkl`	Basic→0, Premium→1

In the app the flow for every prediction is always the same four moves:

1. **Collect** raw human inputs (numbers typed in, words chosen from dropdowns).
2. **Translate** each word into its number using the matching saved encoder's `.transform()` .
3. **Arrange** the values in the exact same column order used in training: `age, income, city, membership` .
4. **Predict** by passing that single row to `model.predict()` .

Why not just type the numbers by hand?
If you hand-code "Paris = 2" and it was actually 3 in training, the model gives wrong answers with
no error message
. Reloading the saved encoder guarantees the numbers match training exactly. Always let the encoder do the translating.

Before You Start (Checklist)

- ✓ Your three files: `model.pkl` , `city_encoder.pkl` , `membership_encoder.pkl` .
- ✓ Python 3.9 or newer installed on your computer.
- ✓ A free **GitHub** account (sign up at `github.com`).
- ✓ A free **Streamlit Community Cloud** account (sign in at `share.streamlit.io` with GitHub).
- ✓ Know the scikit-learn version you trained with (run `pip show scikit-learn`). You'll need it in Step 3.

1 Put all files in one project folder

Create a folder with a simple name like `purchase-predictor`. Move your three saved files into it. You will add two more files (`app.py` and `requirements.txt`) in the next steps.

```
purchase-predictor/
├── app.py                    ← the web app (Step 2)
├── model.pkl                 ← your trained model
├── city_encoder.pkl          ← City translation table
├── membership_encoder.pkl    ← Membership translation table
└── requirements.txt          ← tools to install (Step 3)
```

> **Tip**
> All five files must sit in the same folder. Streamlit Cloud loads them together from your GitHub repository.

2 Write the app (app.py)

Create `app.py` in the folder and paste the code below. Read the numbered comments — they show exactly where the encoder files are loaded and used.

app.py

```
import streamlit as st
import pickle
import pandas as pd

# 1. Load the model AND both encoders (done once when the app starts)
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

with open("city_encoder.pkl", "rb") as f:
    city_encoder = pickle.load(f)

with open("membership_encoder.pkl", "rb") as f:
    membership_encoder = pickle.load(f)

# 2. Page title
st.title("Customer Purchase Predictor")
st.write("Fill in the details and click Predict.")

# 3. Numeric inputs -> typed directly as numbers
age = st.number_input("Age", min_value=18, max_value=100, value=30)
income = st.number_input("Income", min_value=0, value=50000)

# 4. Text inputs -> dropdowns filled from each encoder's known categories.
#    Using .classes_ means the user can ONLY pick values the model has seen.
city = st.selectbox("City", city_encoder.classes_)
membership = st.selectbox("Membership", membership_encoder.classes_)

# 5. Predict button
if st.button("Predict"):
    # 5a. Translate the chosen words into numbers using the SAME saved encoders
    city_num = city_encoder.transform([city])[0]
    membership_num = membership_encoder.transform([membership])[0]

    # 5b. Build one row in the SAME column order used during training
    row = pd.DataFrame(
        [[age, income, city_num, membership_num]],
        columns=["age", "income", "city_encoded", "membership_encoded"]
    )

    # 5c. Predict, and also show the probability
    result = model.predict(row)[0]
    proba = model.predict_proba(row)[0][1]

    if result == 1:
        st.success(f"Likely to purchase  (probability {proba:.0%})")
    else:
        st.info(f"Unlikely to purchase  (probability {proba:.0%})")
```

Where the encoders do their job
Section **1** loads them. Section **4** uses `city_encoder.classes_` to build safe dropdowns of valid options. Section **5a** uses `.transform()` to convert the picked word into the exact number training used. That is the complete role of the encoder files.

Keep the column order fixed
In section 5b the order is `age, income, city_encoded, membership_encoded` — identical to training. If you reorder these, predictions become meaningless. Do not change the order.

3 List the tools (requirements.txt)

Streamlit Cloud starts as an empty machine, so you must tell it what to install. Create `requirements.txt` in the folder. Because the app loads a model and encoders made with scikit-learn, list it — and **pin the same version you trained with** so the saved files load without errors.

requirements.txt

```
streamlit
scikit-learn==1.4.2
pandas
```

> **Version must match**
> Replace `1.4.2` with your own version from `pip show scikit-learn`. A mismatch is the most common cause of "the encoder/model won't load" errors online.

4 Test it on your computer first

Open a terminal (Command Prompt on Windows, Terminal on Mac), move into your folder, and run:

```
pip install -r requirements.txt
streamlit run app.py
```

A browser tab opens with your app. Pick a city and membership, enter age and income, and click **Predict**. Confirm you get a result and no error. Stop the app with `Ctrl + C` when done.

> **Tip**
> If the dropdowns show your real city and membership options, your encoders loaded correctly — that alone confirms the tricky part is working.

5 Upload the project to GitHub

Pick whichever method is easier for you.

Option A — No commands (drag & drop):

1. At `github.com`, click **+** (top-right) → **New repository**.
2. Name it `purchase-predictor`, keep it **Public**, click **Create repository**.
3. Click **uploading an existing file**.
4. Drag in all five files: `app.py`, `model.pkl`, `city_encoder.pkl`, `membership_encoder.pkl`, `requirements.txt`.
5. Click **Commit changes**.

Option B — Git commands:

```
git init
git add .
git commit -m "Deploy purchase predictor"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/purchase-predictor.git
git push -u origin main
```

> **Don't forget the encoder files**
> It is easy to upload `app.py` and `model.pkl` but forget the two `*_encoder.pkl` files. If they are missing, the app crashes with a "file not found" error. Double-check all five files appear in your repository.

6 Deploy on Streamlit Community Cloud

1. Go to `share.streamlit.io` and sign in with GitHub.
2. Click **Create app** → **Deploy a public app from GitHub**.
3. Choose the repo `purchase-predictor`, branch `main`, and set the main file to `app.py`.
4. Click **Deploy**.

Streamlit installs your tools and builds the app (a couple of minutes the first time). When it finishes, your app opens automatically at a public link like `https://purchase-predictor.streamlit.app`.

> **You're live!**
> Share that link with anyone. To update later, just change the files on GitHub — the app rebuilds itself within a minute.

Common Problems & Quick Fixes

What you see	What it means & how to fix it
`FileNotFoundError: city_encoder.pkl` (or membership)	An encoder file wasn't uploaded, or its name differs. Make sure both `*_encoder.pkl` files sit next to `app.py` in the repo.
Error mentioning version / "incompatible" when loading	The scikit-learn version online differs from training. Pin the exact version in `requirements.txt`.
Dropdown is empty or shows odd values	The wrong encoder file was loaded, or it's corrupted. Re-export it from the notebook and re-upload.
Predictions look random or always the same	The input column order doesn't match training, or a word was mapped by hand instead of via `.transform()`. Recheck Step 2, section 5.
`ValueError: y contains previously unseen labels`	A value was sent to an encoder it never saw in training. Because the dropdowns use `.classes_`, this shouldn't happen — make sure you didn't replace them with free text boxes.
`ModuleNotFoundError`	A tool is missing from `requirements.txt`. Add its name and save.

Quick Recap

- ✓ Put `model.pkl` and both encoder files in one folder.
- ✓ Wrote `app.py` that loads the model + encoders and uses `.transform()` on the user's words.
- ✓ Listed tools in `requirements.txt` with the matching scikit-learn version.
- ✓ Tested locally with `streamlit run app.py`.
- ✓ Uploaded all five files to a public GitHub repo.
- ✓ Deployed on Streamlit Community Cloud and got a shareable link.

Your logistic regression model is now a live web app that correctly handles text inputs. 🎉